FTML practical session 1

5 mars 2026

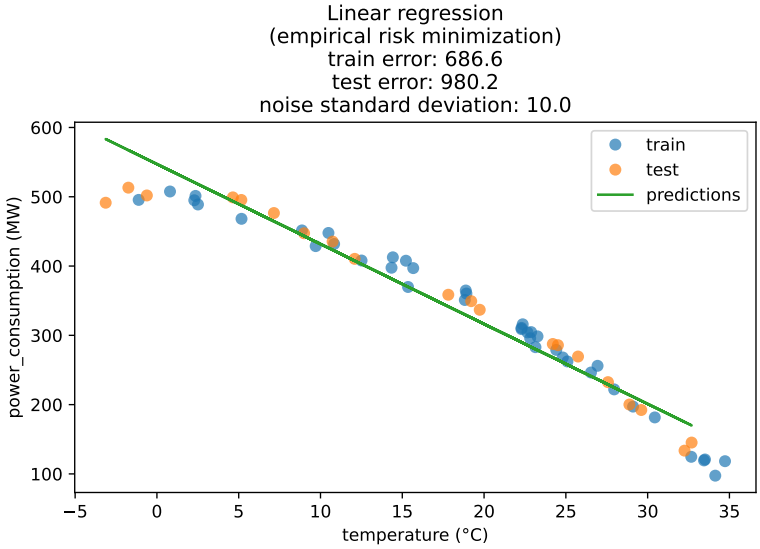

TABLE DES MATIÈRES

1 Train and test sets, empirical risk, overfitting 3
1.1 Introduction . . . 3
1.2 Computation of the empirical risk for random parameters . . . 3
1.3 Linear regression : computation of the optimal parameters . . . 3
1.4 Polynomial regression and overfitting . . . 4
2 Experimenting with the law of large numbers and array programming 7
2.1 More Python profiling . . . 7
3 Example of risks and Bayes risks 8
3.1 Example empirical risks . . . 8
3.1.1 Problem 1 : penalty shootout . . . 8
3.1.2 Problem 2 : prediction of the number of spotify streams . . . 8
3.2 Bayes estimators and Bayes risks . . . 8
4 Overfitting and variance of the test error 9
4.1 Standard deviation of the test error . . . 9

INTRODUCTION

The goal of this practical session is to experiment with some statistical or probabilistic concepts that are important machine learning : namely, empirical risks, overfitting, and the law of large numbers. During the following lectures, we will study and formalize the definitions of the different types of risks in more depth.

You can do the different parts in whatever order. You do not have to finish everything during the session.

INSTALLATION AND PYTHON

Environments

To install libraries, you can use virtual environments.
https://docs.python.org/3/library/venv.html

You can use for instance **uv** as a package manager. uv will use **pyproject.toml** to install the virtual environment.
https://github.com/astral-sh/uv.

When using uv, a pyton script can be run by typing "uv run script.py" in a terminal.

The list of libraries used in the practical sessions is also in **requirements.txt**. You can use it to install all libraries directly with pip, e.g. with **pip instlall -r requirements.txt**

Demos

In the **demos/** subfolder of this session, you can find a couple of simples demo files to use matplotlib, numpy (if needed).

Operations in Python

Time complexity of elementary operations in python :
https://wiki.python.org/moin/TimeComplexity

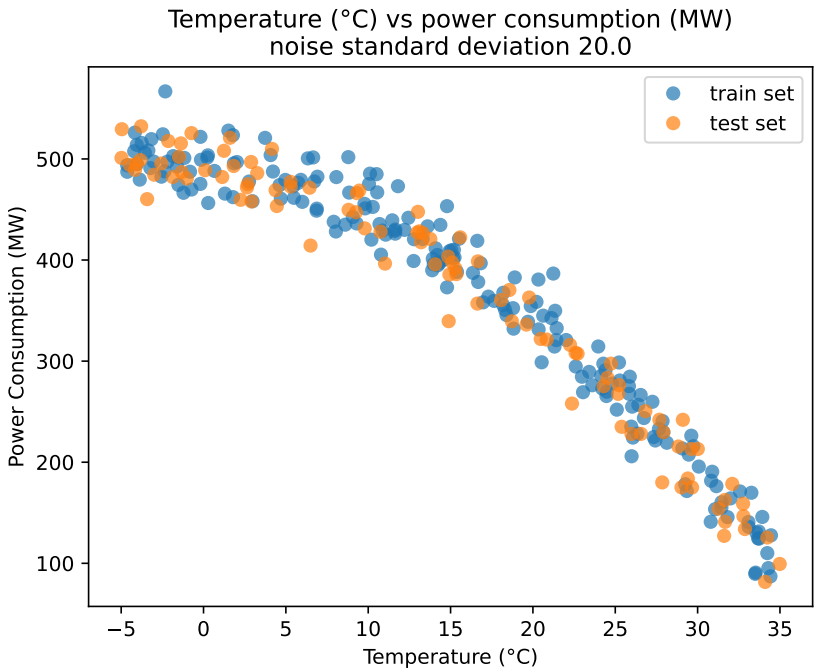

FIGURE 1 – Example dataset. You can use different noise magnitudes and different number of samples.

1 TRAIN AND TEST SETS, EMPIRICAL RISK, OVERFITTING

1.1 Introduction

In this exercise we work with the dataset used in lecture 1b (electricity production).

1.2 Computation of the empirical risk for random parameters

cd practical_sessions/tp_01_train_test_overfitting/1_empirical_risk/

— Generate some data using **main_create_data.py**. You can choose the amplitude of the noise by setting STD_NOISE in **constants.py**. This noise represents the existence of **hidden variables**, as discussed during the lecture. The images are generated in the **images/datasets/** folder. An example output is shown in figure 1

— Launch **main_random_params.py** in order to test several random values for θ and b by evaluating their empirical risk. To do so, you will need to fix empirical_risk() inside **utils.py**. Note that you can know whether your result is correct thanks to the ground truth (mse()).

Remark : In VSCode, you need to execute the python script from the correct location (namely from **1D_linear_regression**) in order for the paths to properly work.

1.3 Linear regression : computation of the optimal parameters

Launch **main_optimal_params.py** in order to evaluate the empirical risk for the optimal values of the parameters, as found during the lecture 1b. To do so, you will need to fix compute_optimal_params() inside **utils.py**. Plot the predictions obtained by the optimal linear estimator. You should observe a plot similar to Figure 2.

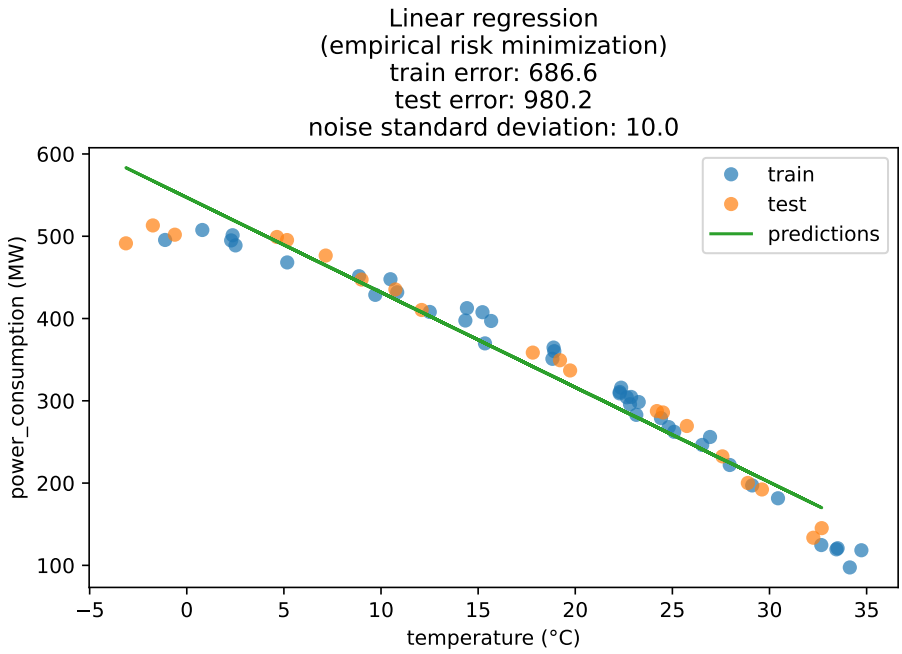

FIGURE 2 – Linear regression

1.4 Polynomial regression and overfitting

Now, we will experiment with different models and use **polynoms** to try to fit the data. Here, our objective is to study the interaction between the number of samples, the amount of noise in the data, overfitting, and importantly, the **degree** of the polynom.

— Before implementing the polynomial fit or looking at the images, try to anticipate the results. What behavior do you expect ?

— Implement a polynoial regression and observe this interaction. You should obtain results similar to figures 3, **??**, 5, 6.

You can use function from numpy like polyfit and polyval.

Polynomial regression can actually be written as a linear regression, but in a different space. Can you think of which one, and determine how to write it ?

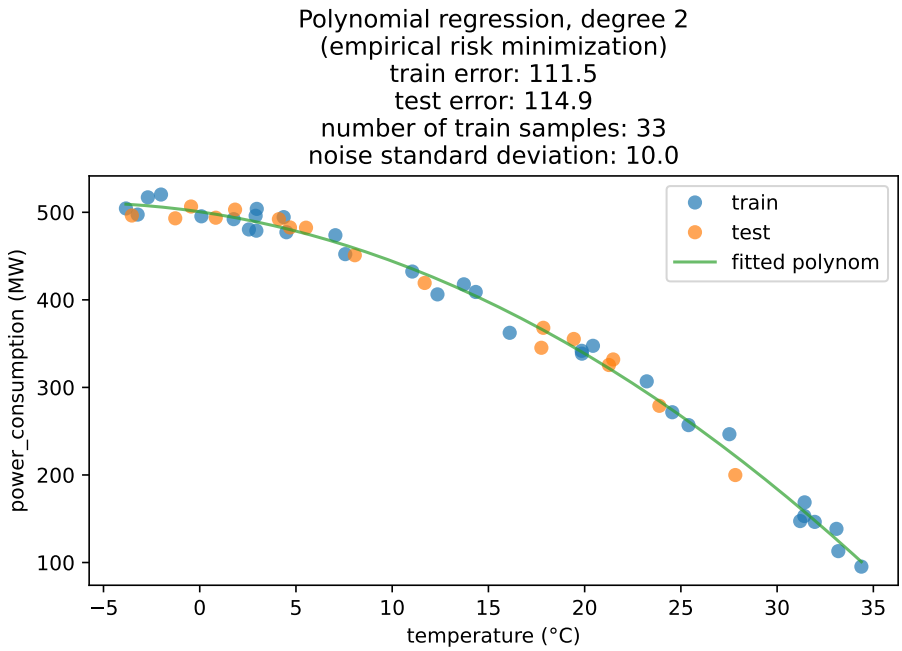

FIGURE 3 – Relevant polynom for these data

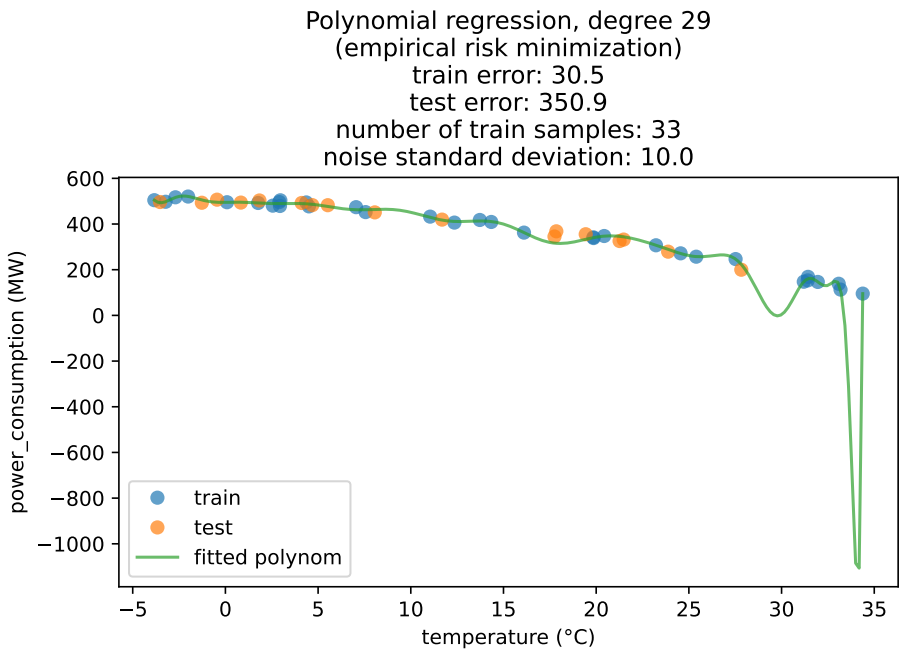

FIGURE 4 – Overfitting, when the degree of the polynom is larger, and not balanced by sufficient number of training samples

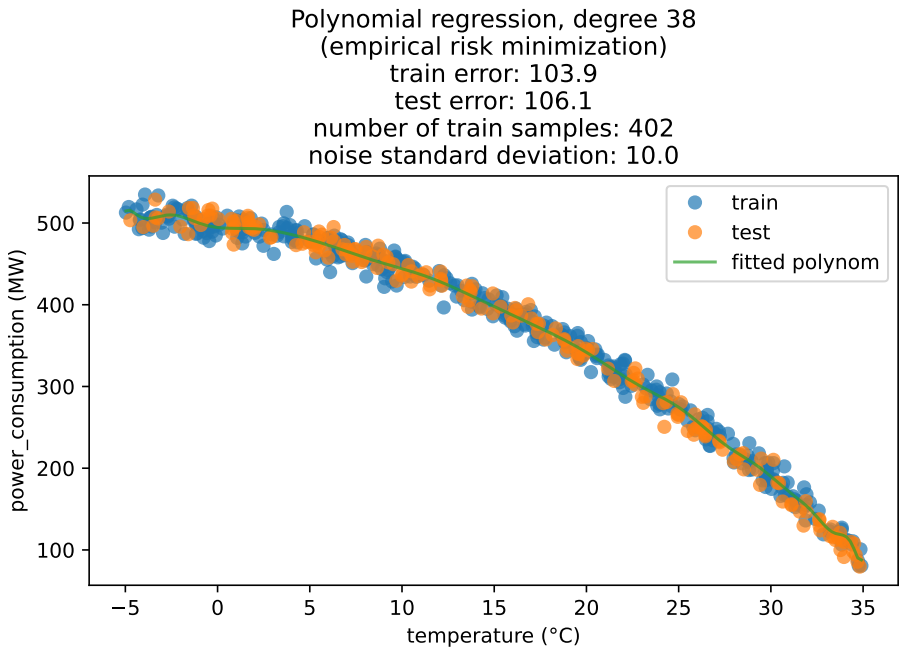

FIGURE 5 – Although the degree of the polynom is high, the number of samples is high enough to prevent overfitting.

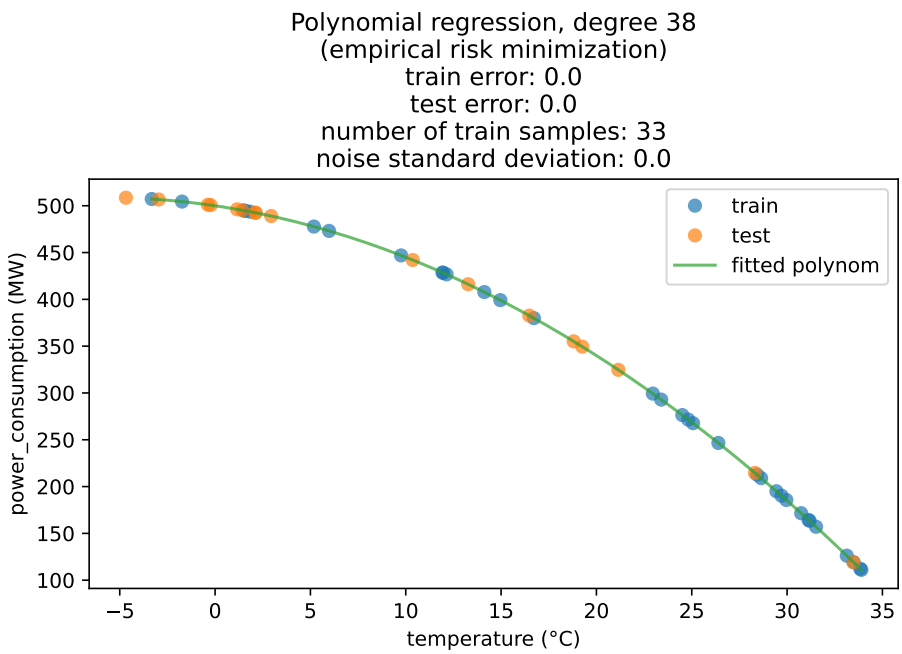

FIGURE 6 – Without noise, there is no overfitting!

2 EXPERIMENTING WITH THE LAW OF LARGE NUMBERS AND ARRAY PROGRAMMING

Let us consider the same variable as in exercice P_3 : $Z_2 = Z_1$ and is Z_1^2 is a uniform law in $[1, 2]$. We have seen that $E[Z_2] = 7/3$. Hence, according to the law of large numbers, the empirical average of n draws of this variable converges in probability to this expected value.

In **2_arrays_programming/law_of_large_numbers.py**, the function **empirical_average_loop** computes the empirical average with a for loop

— Edit the function **empirical_average_array()** in order to use numpy and array operations to perform the same computation in an optimized way, only using array operations and without a loop.

— Compare the speed of the methods by monitoring the **profile.prof** profiling file, for instance using **snakeviz profile.prof**.

References :

— https://en.wikipedia.org/wiki/Array_programming

— https://jiffyclub.github.io/snakeviz/

Note that it is also possible to save the profiling to a text file.

2.1 More Python profiling

It is also possible to profile individual lines :
https://pypi.org/project/line-profiler/

Or to profile memory usage (this package is not maintained anymore, though) :
https://pypi.org/project/memory-profiler/

3 EXAMPLE OF RISKS AND BAYES RISKS

The goal of this exercise is to compute some practical exemples of empirical risks, risks, and to introduce the notion of Bayes risk, a theoretical notion that we will study more extensively during the couse.

3.1 Example empirical risks

For the two following problems : sample a small number (e.g. $4, 5$) of samples from the simulation and compute empirical risks of the estimators of your choice.

3.1.1 *Problem 1 : penalty shootout*

We represent a football penalty shootout. $X \in \{0, 1\}$ is the variable representing that team 1 shoots first. $Y \in \{0, 1\}$ is the variable representing the fact that team 1 wins. We assume that :

- X is uniformly distributed.
- If $X = 1$, Y follows a Bernoulli law of parameter 0.6. If $X = 0$, Y follows a law of parameter 0.4.
- l is the $0 - 1$ loss (1 if there is a mistake, 0 otherwise)

3.1.2 *Problem 2 : prediction of the number of spotify streams*

A music label is interested in predicting the number of streams of an artist, as a function of the investment. We will consider that the investment is represented by the number of persons who work with the artist during the production. This variable is noted X. More precisely, we predict the number of streams of the song on Spotify, noted Y, during the first week after release, as a function of X. We assume that :

- $X - 1 \in \mathbb{N}$ follows a binomial law of parameters $n_X = 20$ and $p_X = 0.2$. Hence, $X > 0$.
- Given a value x of X, Y follows a binomial law of parameters $n_Y(x) = 3^x$ and $p_Y(x) = 0.5$.
- l is the squared loss.

3.2 Bayes estimators and Bayes risks

Under some simple hypotheses, for a given supervised learning problem, there exists an optimal estimator f^* called the **Bayes estimator** : its definition is that is has the smallest possible the "generalization error", given a distribution ρ. The generalization error is the expected value of the error, given the distribution of the data. The generalization error of the Bayes estimator is also called the **Bayes risk**.

For the two problems, find the Bayes estimator and the Bayes risk.

- For problem 1, the Bayes risk is 0.4.
- For problem 2, the Bayes risk is around 627.

4 OVERFITTING AND VARIANCE OF THE TEST ERROR

In this exercice we experimentally study the amount of overfitting and the empirical variance of the test error as a function of the number of samples used in the datasets. To do so, we use a linear regression problem.

4.1 Standard deviation of the test error

In statistical learning, we consider that datasets are sampled from random variables. Hence, any quantity that is computed from a dataset is also a random variable. For instance, the **test error**, which is the empirical risk on a **test set** (a dataset that was not used at all for training) is a real random variable, that has a standard deviation. Intuitively, and according to the law of large numbers, the test error converges to the generalization error when the number of samples becomes large enough.

Using scikit-learn, optimize a linear estimator on the train set, and monitor the standard deviation of the test error as a function of the number of samples used in the test set.

You can find a template file to edit **exercice_4.py** in **exercice_2_statistics/**. It uses the LinearRegression class from scikit. You should observe a plot like Figure 7.

You may also experiment with the number of samples used in the train set. This parameter will also influence the test error.

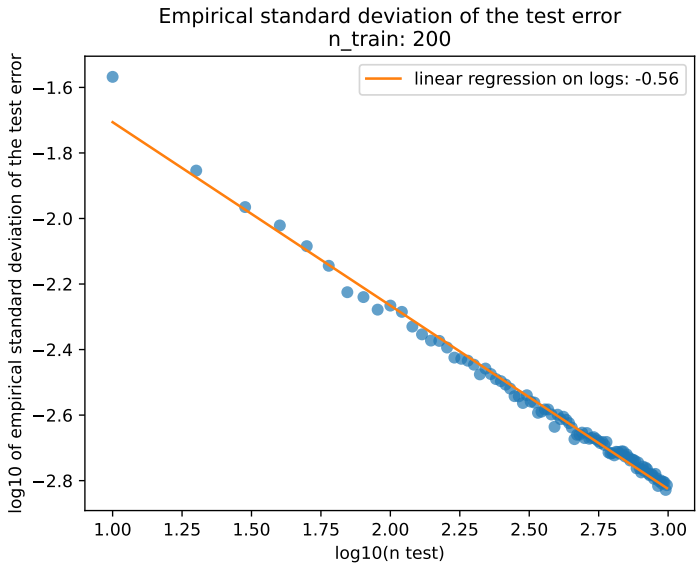

FIGURE 7 – This behavior is typical of the law of large numbers. Why do we observe a linear decrease of the standard deviation, when represented in log scale ?